

SHOPEZ

E-commerce Application • Comprehensive Technical Architecture & Documentation

1. Platform Overview

ShopEZ provides a comprehensive and user-friendly shopping experience with a wide range of products available in its catalog. Each product includes detailed information such as description, customer reviews, pricing, and available discounts, helping users make informed purchase decisions.

Every product listing contains a "Shop Now" button, allowing users to quickly proceed with the purchasing process. After selecting a product, users are taken to the Order Details Page, where they can enter important information such as shipping address, preferred payment method, and specific product requirements. The platform ensures a secure and efficient checkout process, handling all user data and payment information safely to provide a smooth transaction experience. Once the order is placed successfully, users receive an order confirmation notification and can view a detailed order summary.

2. Core Entities

User

Represents individuals who are registered on the ShopEZ platform. Each user has a unique account created during registration, storing personal information such as name, email address, password, and delivery address. Users can log in securely, browse available products, search items, view details, add products to their cart, place orders, track order status, and view order history.

Admin

Represents the administrator who manages and controls the entire ShopEZ platform. The admin has privileged access to system functionalities and is responsible for maintaining the product catalog, managing banner images, product categories, and listings. Admins also monitor user activity, manage orders, and ensure smooth platform operations.

Products

Represent all the items available for purchase. Each product contains detailed info (name, description, price, category, stock availability, discount offers, images). The product entity is continuously updated by the admin to ensure accurate pricing and availability.

Cart & Orders

Cart: A temporary storage area where users can store selected products before checkout. Each cart entry is tied to a specific User ID independently. Users can update quantities or remove items.

Orders: Represent all confirmed purchases. Contains product information, quantity, total price, payment method, payment status, shipping address, and order status. Used to track delivery and purchase history.

3. Database & ER Model

Entity Relationship Workflow: The database follows a normalized structure to reduce data redundancy, using foreign keys to maintain relationships and ensure data integrity.

- The **Admin** manages product categories and products.
- Each **Category** contains multiple products.
- The **Product** entity stores details such as title, description, price, discount, images, sizes, and category.
- A **User** can add multiple products to the cart and place multiple orders.
- Each **Order** can contain multiple Order Items.
- **Order Item** links products with orders and stores quantity, size, and price.

4. Technical Architecture

The SHOPEZ E-commerce Application is designed using a three-tier architecture consisting of the Frontend, Backend, and Database. This architecture separates the presentation layer, application logic, and data storage, making the system modular, scalable, secure, and easy to maintain.

4.1 Frontend Architecture

The frontend is the presentation layer allowing users and admins to interact through an intuitive GUI. It is responsible for displaying data, collecting input, validating forms, and communicating with the backend via RESTful APIs.

- **Component-Based:** Designed using reusable components for better maintainability and lower code duplication.
- **Validation:** Client-side validation for email formats, passwords, required fields, and files.
- **Responsiveness:** Adapts consistently across desktops, laptops, tablets, and smartphones.
- **State Management:** Maintains logged-in info, cart contents, and selected products smoothly.

4.2 Backend Architecture

The backend forms the business logic layer. It processes requests, performs validations, executes rules, and connects to the database. It follows a layered approach: routing, controllers, services, middleware, and models.

- **API Layer:** RESTful APIs for User Auth, Products, Categories, Cart, Wishlist, Orders, Payments, and Inventory.
- **Business Logic:** Handles tax calculations, stock updates, promotional offers, and checkout logic safely separated from routing logic.
- **Security:** Authentication via encrypted passwords and secure tokens. Role-based authorization isolates customer and admin privileges.
- **Middleware & Error Handling:** Validates tokens, handles CORS, logs requests, and manages centralized exceptions with proper HTTP status codes.
- **Optimization:** Uses efficient database queries, pagination, caching, lazy loading, and async request handling to support high traffic.

4.3 Database Architecture

The storage layer maintains all data securely and efficiently, designed for high availability and scalability.

Collection / Table	Key Responsibilities & Fields
User	User ID, Full Name, Encrypted Password, Address, Role, Account Status.
Product	Product ID, Name, Category, Price, Stock, Images, Availability Status.
Cart	Cart ID, User ID, Product ID, Quantity, Date Added. (Auto-updates)

Collection / Table	Key Responsibilities & Fields
Order	Order ID, Customer ID, Products, Total Amount, Payment Status, Delivery Date.

Security & Scalability: Database security includes password encryption, access control, audit logging, and protection against injection attacks. The architecture supports future growth via indexing, query optimization, replication, and backup strategies.

4.4 Overall Workflow

1. The user accesses the application through the frontend interface.
2. The frontend sends API requests to the backend based on user actions.
3. The backend authenticates the request and executes business logic.
4. The backend retrieves or updates data securely in the database.
5. The database returns the requested information to the backend.
6. The backend formats the response and sends it to the frontend.
7. The frontend dynamically updates the UI, providing real-time feedback without page reloads.